



LogPress

Week 4 주간 보고

Role A · Role C · Role D 통합 진행 보고



LogPress Week3

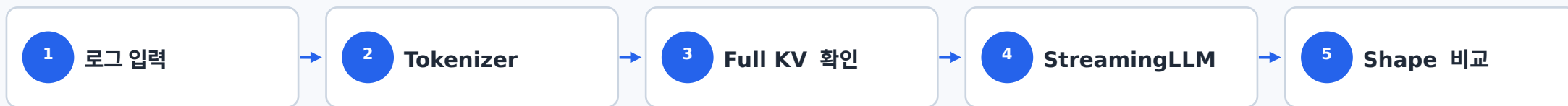
StreamingLLM 재현 진행 공유

Tokenizer → Model Forward → KV Cache → StreamingLLM 압축 확인

보고 기준 : 0.5 압축 smoke test 완료 · 0.7/0.9 는 동일 코드 경로로 확장 가능

이번 주 목표

목표는 최종 성능평가가 아니라 baseline 코드 경로를 실제로 통과시키는 것



확인한 범위

- Qwen2.5-3B-Instruct 로드
- 외부 로그 문자열 토큰화
- DynamicCache 의 K/V shape 확인
- StreamingLLMPress 0.5 적용 후 KV 길이 감소 확인

아직 최종 평가는 아님

- Thunderbird 1 시간 window 전체 평가 전
- 이상 / 원인 라인 보존율 산출 전
- 현재는 baseline 재현과 디버깅 단계

StreamingLLM 0.5 재현 결과

KV Cache 의 sequence length 가 실제로 줄어드는지 확인

Full KV



131 tokens

[1, 2, 131, 128]

약 50% 감소

StreamingLLM 0.5



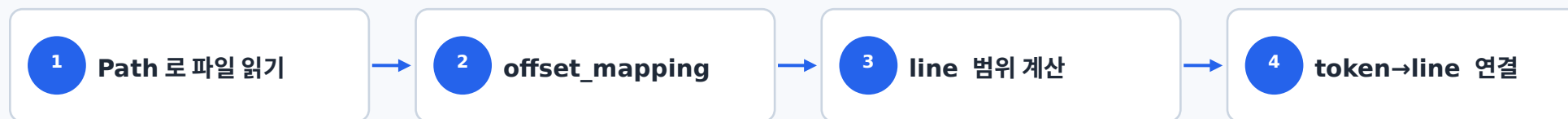
65 tokens

[1, 2, 65, 128]

바뀐 차원은 seq_len 뿐

외부 로그 파일 입력으로 확장

하드코딩 문자열 대신 thunderbird_sample.txt 를 읽는 구조로 전환



현재 확인된 출력 예시

```
line 1 -> token [0, 1]
line 2 -> token [2, 3, 4, 5, 6, 7]
line 3 -> token [8, ..., 18]
line 4 -> token [19, ..., 31]
line 5 -> token [32, ..., 37]
```

의미

각 로그 줄이 어떤 token index 로 변환되는지 확인했다 .
다음 단계에서 StreamingLLM 이 남긴 token index 와 비교하면
KEEP / PARTIAL / DROP 을 계산할 수 있다 .

압축률 확장 계획

0.7 / 0.9 는 같은 코드 경로에서 `compression_ratio` 값만 바꾸는 방식



주의해서 표현할 점

이번 주에는 0.5 결과를 실제로 확인했고 , 0.7/0.9 는 같은 코드에서 파라미터만 바꿔 확장 가능한 상태까지 정리했다 . 실제 수치는 다음 실행에서 기록한다 .

남은 작업

내일 이어서 할 일

1

line 별 token index 를 리스트로 저장

현재는 출력까지 확인 . 다음은 데이터 구조로 저장 .

2

StreamingLLM keep token index 계산

`n_sink=4` 와 `compression_ratio` 계산식 적용 .

3

KEEP / PARTIAL / DROP 출력

각 line 의 token 중 몇 개가 살아남았는지 계산 .

4

Thunderbird 2k 샘플에 적용

하드코딩 샘플을 실제 CSV 기반 로그 입력으로 확장 .

이번 주 결과물은 “최종 성능 수치” 가 아니라 , **baseline** 재현을 위한 실행 경로 확보다 .

Week 3-4 정리

Week 5 계획

Tokenizer · Tensor 학습 정리와 Thunderbird 기반 LogPress 설계 준비

실제 구현 연결 전 기초 정리

5 주차 : Module + LogPress 구조

핵심 메시지

이번 주는 Attention-KV-LogPress 를 실제 연결한 주가 아니라 ,
그 연결을 가능하게 하는 Tokenizer/Tensor 기반을 다진 주입니다 .

이번 주 실제로 한 일

과장 없이 실제 진행한 범위만 정리

Python 기초 보강

for / zip / enumerate / dict / 함수 / class / self
/ 상속 / dataclass 등
라이브러리 코드 읽기에 필요한 문법을 정리

Tokenizer 학습

AutoTokenizer, input_ids, attention_mask,
encode/decode, padding/truncation,
offset_mapping 을 실습
→ line-token mapping 의 기반 확보

Tensor 문법 학습

shape, slicing, reshape/view,
transpose/permute, unsqueeze/squeeze,
cat/stack, matmul, softmax, topk,
index_select 까지 진행

이번 주 결론

- LogPress 구현 자체보다는 , 구현을 읽고 짤 수 있게 만드는 기초를 다짐
- Attention/KV 와의 연결은 개념 설명 수준이며 실제 KV 삭제 실험은 아직 진행 전
- Colab 실습 노트는 GitHub part_C/week3-4 경로에 올리는 흐름으로 정리

Tokenizer 학습 내용

텍스트를 token 위치까지 추적하는 기반



이해한 것

- AutoTokenizer.from_pretrained() 로 모델에 맞는 tokenizer 객체를 로드
- tokenizer(text) 의 결과에서 input_ids 와 attention_mask 를 dict 처럼 확인
- convert_ids_to_tokens() 와 decode() 의 차이 확인
- padding/truncation 은 batch 길이와 max_length 를 맞추는 과정으로 이해
- return_offsets_mapping=True 로 token 이 원문 몇 번째 문자에서 왔는지 확인

LogPress 와 연결되는 지점

offset_mapping + line range 를 이용해
각 token 이 어느 로그 line 에 속하는지 계산할
수 있음 .

5 주차에는 Thunderbird 로그에 이 구조를
실제 적용하는 것이 목표 .

Tensor 학습 내용

KVPress 코드 읽기에 필요한 shape 감각 확보

[B, H, T, Dh]

Batch · Head · Token · Head dimension

Shape 조작

slicing: keys[:, :, -6:, :]
reshape/view: [B,T,D] → [B,T,H,Dh]
transpose/permute: [B,T,H,Dh] → [B,H,T,Dh]

Attention 계산 연결

$Q @ K^T \rightarrow [B,H,T,T]$
 $\text{softmax}(\text{dim}=-1) \rightarrow \text{key 축 기준 비율화}$
각 query 별 attention 합은 1

Token 선택 연결

$\text{topk}() \rightarrow \text{중요한 token index 선택}$
 $\text{index_select}() \rightarrow \text{공통 index 로 KV 선택}$
 $\text{gather}() \rightarrow \text{head 마다 다른 index 선택 예정}$

아직 남은 Tensor 마무리

- $\text{gather}()$ 를 이용해 head 별 다른 token 선택
- dtype / device 확인
- $\text{topk} \rightarrow \text{gather} \rightarrow \text{selected KV 흐름 종합 실습}$

정확한 현재 위치

이번 주 성과와 아직 하지 않은 일을 구분

완료 / 진행됨

- Tokenizer 핵심 API 이해
- offset_mapping 으로 원문 위치 확인
- Tensor shape 와 기본 연산 실습
- topk/index_select 까지 KV 선택의 기초 학습
- Colab 노트 정리 및 GitHub 업로드 흐름 확인

아직 아님

- Attention-KV-LogPress 실제 연결 구현
 - 실제 KV cache 삭제 또는 압축 실험
- Thunderbird 기반 LogPress 완성
- Nezha RCA 평가나 원인 보존 검증
 - 복잡한 희귀도 / 템플릿 기반 scoring

따라서 5 주차는 “구현 기반 → 실제 LogPress 구조 설계” 로 넘어가는 주로 잡는 것이 적절합니다 .

Thunderbird 데이터셋 기준 LogPress 구조

데이터셋 준비가 아니라, 준비된 Thunderbird 를 활용하는 단계



5 주차에서 만들 구조

- Thunderbird 로그 window 를 입력으로 사용
 - 각 로그 line 의 문자 범위 [start, end] 계산
- HF tokenizer 의 offset_mapping 으로 token_idx → line_idx 테이블 생성
- line 단위 중요도 점수를 token 점수로 전달
- token score 에 topk 를 적용해 keep/drop 후보를 만든다

처음 scoring 은 단순하게

처음부터 복잡한 원인 추론을 하지 않는다 .

예시 :

ERROR line: 높게

WARN line: 중간

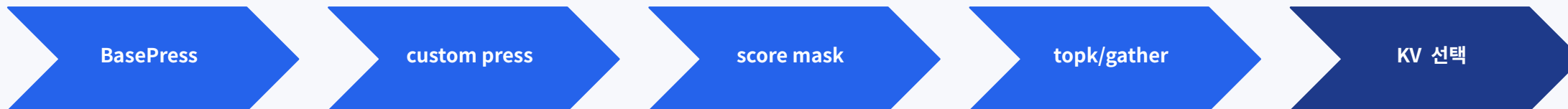
그 외 : 낮게

목표는 “ line → token → topk ” 가 실제로 돌아가는지 확인하는 것 .

KVPress 재진입과 백업 경로

본선은 BasePress 상속, 백업은 병렬로 구조만 확인

본선 흐름



이번 주 확인할 코드 포인트

- BasePress.compress() 의 입력과 반환 구조 파악
- keys / values shape 와 token 축 위치 확인
- module.layer_idx 를 이용한 레이어 조건 분기 가능성 확인
- line score 를 token score 로 바꿔 기존 press 구조에 넣을 수 있는지 확인

백업 경로는 병렬 조사

- 1 순위 : BasePress 상속으로 구현
- 2 순위 : kvpress fork 에 custom press 추가
- 3 순위 : transformers Cache 직접 제어

5 주차에는 구현보다 “막히는 지점과 우회 가능성” 을 확인

5 주차 할 일

이번 주 계획 : Module 을 끝내고 Thunderbird 기반 LogPress 구조를 시작

순서	작업	목표
1	Tensor 마무리	gather / dtype / device / topk→gather 로 selected KV 흐름 정리
2	nn.Module 학습	__init__, forward, Linear, model(x), parameters, named_modules, hook
3	SimpleAttention 구현	Q/K/V projection → reshape → attention score → softmax 흐름 직접 작성
4	KVPress 코드 재진입	BasePress.compress 구조와 layer_idx, keys/values shape 확인
5	Thunderbird LogPress 구조	line range → offset_mapping → token_to_line → line score → token topk
6	백업 경로 병렬 확인	fork custom press / Cache 직접 제어 가능성 조사

5 주차 성공 기준 : prepared Thunderbird 를 사용해 “로그 line → token index → score → top-k keep 후보” 가 코드로 돌아가는 것 .



D

보존율 채점기 구현

Preservation-Rate Scorer — 원인 줄 생존 채점

무엇을, 왜 만들었나

정의 KV 캐시 압축 후, 장애 원인 줄 (cause line) 이 살아남았는지를 세어 **보존율**을 산출하는 채점 도구

Tier-1 · 보존율 (안전판)

원인 줄이 살았는가 . 모델 정확도와 독립적으로 측정 → 문제가 아무리 어려워도 결과가 나온다 .

Tier-2 · RCA delta (결정타)

압축 후 모델이 원인을 더 잘 맞히는가 . 강력하지만 , 문제 난도가 높으면 무너질 위험 (floor effect).

핵심 논거

Tier-2 가 floor 로 무너져도 Tier-1 결과만으로 논문이 성립한다 . 그래서 채점기를 여유 구간에 미리 구현해 둔다 .

채점기가 하는 일 — 세 단계

1

세기

원인 줄을 하나씩 보며 압축 후 살아남은 것만 카운트 → 살아남은 수 ÷ 전체 = 보존율

2

층화 (골라내기)

전체 평균만 보면 'ERROR 라서 살린 것' 이란 반박에 막힘 . ERROR 아닌 원인 . 중간 위치 원인만 따로 분리

3

세 번 집계

같은 세기를 overall · non_error · middle 세 층에 각각 적용 → 보존율 세 개 산출

보존율 산출 (핵심)

```
def rate(target):  
    preserved = 0  
    for c in target:  
        if keep[c]:  
            preserved += 1  
    return preserved / len(target)
```

```
overall    = rate(cause)  
non_error  = rate(non_error)  
middle     = rate(middle)
```

검산 — 손계산과 코드 일치

로그 10 줄짜리 합성 케이스 . 원인 줄 4 개를 손으로 세어 코드 출력과 대조 .

원인 줄	생존	ERROR	위치
2	생존	O	0.30
5	사망	X	0.55
7	생존	X	0.80
8	생존	O	0.90

overall

0.75

4 개 중 3 개 생존

non_error

0.50

대상 5·7 중 7

middle

0.50

대상 2·5 중 2

결과 코드 출력 0.75 / 0.50 / 0.50 — 손계산과 정확히 일치 . 채점 로직 검증 완료 .

비교 · 판정 — 남들보다 나은가

McNemar 검정

두 방법이 갈린 줄만 센다. 비긴 줄 (둘 다 생존 / 사망) 은 우열 정보가 없어 버림. 갈린 것이 한쪽으로 치우칠수록 ' 진짜 우세 '.

BH-FDR 보정

baseline 4 개 × 총 3 개 = 12 번 비교. 여러 번 하면 우연히 이기는 게 섞이므로, p 값을 순위별로 밀어 올려 걸러냄.

Tier-1 판정

hard-case(non_error·middle) 에서 4 개 baseline 전부 보정 후에도 유의하게 이기면 → PASS. 이것이 논문의 핵심 결과.

비교 대상 (baseline)

StreamingLLM

SnapKV

PyramidKV

H2O

+ LogPress (우리 방법)

정리 문서 — 발표 / 제출용

본문은 프로젝트 표준 용어로 , 부록에서 평이한 설명으로 — 두 층으로 구성 .

목적

Tier-1 안전판 논거 — 왜 이 채점기가 논문을 세우나

Interface

cause / keep / is_error / pos 입출력 명세 + C
의존성

검증

합성 데이터 0.75 / 0.50 / 0.50 대조표

아키텍처

3 개 컴포넌트 : 재료 불러오기 · 채점 · 비교 · 판정

Scoring

보존 판정 규칙 (rule=any) + 총화 로직 (완료분)

부록

표준 용어 ↔ 평이한 설명 대응표

Thunderbird 로그 파싱

원본 로그를 필드별로 쪼개 정리하는 전처리 . (원본을 보기 좋게 구조화한 수준)

원본 로그 (한 줄)

```
- 1131564688 2005.11.09 dn228 Nov 9 ... component:  
content...
```



구조화된 필드

```
label · timestamp · date · user · time · location ·  
component · content
```

한 일

- 각 로그 줄을 공백 기준으로 필드 분리 → 딕셔너리로 구조화
- 하드케이스 원인 줄에 원문 content 를 매칭해 grid_with_content.csv 생성
- 채점기의 정답지 (cause·is_error·pos) 로 쓰일 정리된 CSV 산출

Week 4 완료 · 다음

완료 ✓

- ✓ 보존을 채점기 본체 (세기 · 층화 · 집계)
- ✓ 합성 데이터 검산 (0.75 / 0.50 / 0.50)
- ✓ McNemar + BH 비교 · 판정 로직
- ✓ 코드 정리 (역할별 셀 구성)

다음 →

→ Ablation(제거 실험) 설계

부품을 하나씩 빼보며 '이게 진짜 우위의 원인인가'를 확인하는 실험. 지금은 설계 (계획) 만 — 무엇을 · 어떤 압축률 · 지표로 비교할지 표로 못 박아 둔다.

실측 일정

8 주차 ~ 10 주차 예정

C 의 keep mask 인계 후 실측 진입

다음 주 계획 — 역할별

A

Role A

- line 별 token index 자료구조 저장
- StreamingLLM keep token index 계산
- KEEP / PARTIAL / DROP 출력
- Thunderbird 2k 샘플에 적용

C

Role C

- Tensor 마무리 (gather / topk→gather 흐름)
- nn.Module · SimpleAttention 구현
- KVPress BasePress.compress 재진입
- Thunderbird LogPress 구조
(line→token→topk)

D

Role D

- Ablation(제거 실험) 설계 — 비교표 못 박기
- 실측은 8~10 주차 예정
- C 의 keep mask 인계 후 진짜 판정